

Running scripts

A Python script is a file intended for standalone execution, e.g., with `python <script>.py`. Using `uv` to execute scripts ensures that script dependencies are managed without manually managing environments.

!!! note

If you are not familiar with Python environments: every Python installation has an environment that packages can be installed in. Typically, creating `[_virtual_ environments]` (<https://docs.python.org/3/library/venv.html>) is recommended to isolate packages required by each script. `uv` automatically manages virtual environments for you and prefers a `[declarative](#declaring-script-dependencies)` approach to dependencies.

Running a script without dependencies

If your script has no dependencies, you can execute it with `uv run`:

```
print("Hello world")
```

```
$ uv run example.py
Hello world
```

Similarly, if your script depends on a module in the standard library, there's nothing more to do:

```
import os

print(os.path.expanduser("~/"))
```

```
$ uv run example.py
/Users/astral
```

Arguments may be provided to the script:

```
import sys

print(" ".join(sys.argv[1:]))
```

```
$ uv run example.py test
test
```

```
$ uv run example.py hello world!
hello world!
```

Additionally, your script can be read directly from stdin:

```
$ echo 'print("hello world!")' | uv run -
```

Or, if your shell supports here-documents:

```
uv run - <<EOF
print("hello world!")
EOF
```

Note that if you use `uv run` in a *project*, i.e., a directory with a `pyproject.toml`, it will install the current project before running the script. If your script does not depend on the project, use the `--no-project` flag to skip this:

```
$ # Note: the `--no-project` flag must be provided _before_ the script name.
$ uv run --no-project example.py
```

See the projects guide for more details on working in projects.

Running a script with dependencies

When your script requires other packages, they must be installed into the environment that the script runs in. `uv` prefers to create these environments on-demand instead of using a long-lived virtual environment with manually managed dependencies. This requires explicit declaration of dependencies that are required for the script. Generally, it's recommended to use a project or inline metadata to declare dependencies, but `uv` supports requesting dependencies per invocation as well.

For example, the following script requires `rich`.

```
import time
from rich.progress import track

for i in track(range(20), description="For example:"):
    time.sleep(0.05)
```

If executed without specifying a dependency, this script will fail:

```
$ uv run --no-project example.py
Traceback (most recent call last):
  File "/Users/astral/example.py", line 2, in <module>
    from rich.progress import track
ModuleNotFoundError: No module named 'rich'
```

Request the dependency using the `--with` option:

```
$ uv run --with rich example.py
For example: _____ 100% 0:00:01
```

Constraints can be added to the requested dependency if specific versions are needed:

```
$ uv run --with 'rich>12,<13' example.py
```

Multiple dependencies can be requested by repeating with `--with` option.

Note that if `uv run` is used in a *project*, these dependencies will be included *in addition* to the project's dependencies. To opt-out of this behavior, use the `--no-project` flag.

Creating a Python script

Python recently added a standard format for inline script metadata. It allows for selecting Python versions and defining dependencies. Use `uv init --script` to initialize scripts with the inline metadata:

```
$ uv init --script example.py --python 3.12
```

Declaring script dependencies

The inline metadata format allows the dependencies for a script to be declared in the script itself.

uv supports adding and updating inline script metadata for you. Use `uv add --script` to declare the dependencies for the script:

```
$ uv add --script example.py 'requests<3' 'rich'
```

This will add a `script` section at the top of the script declaring the dependencies using TOML:

```
# /// script
# dependencies = [
#     "requests<3",
#     "rich",
# ]
# ///
```

```
import requests
from rich.pretty import pprint

resp = requests.get("https://peps.python.org/api/peps.json")
data = resp.json()
pprint([(k, v["title"]) for k, v in data.items()][:10])
```

uv will automatically create an environment with the dependencies necessary to run the script, e.g.:

```
$ uv run example.py
[
  ('1', 'PEP Purpose and Guidelines'),
  ('2', 'Procedure for Adding New Modules'),
  ('3', 'Guidelines for Handling Bug Reports'),
  ('4', 'Deprecation of Standard Modules'),
  ('5', 'Guidelines for Language Evolution'),
  ('6', 'Bug Fix Releases'),
  ('7', 'Style Guide for C Code'),
  ('8', 'Style Guide for Python Code'),
  ('9', 'Sample Plaintext PEP Template'),
  ('10', 'Voting Guidelines')
]
```

!!! important

When using inline script metadata, even if ``uv run`` is [used in a `_project_`](../concepts/projects/run.md), the project's dependencies will be ignored. The ``--no-project`` flag is not required.

uv also respects Python version requirements:

```
# /// script
# requires-python = ">=3.12"
# dependencies = []
# ///

# Use some syntax added in Python 3.12
type Point = tuple[float, float]
print(Point)
```

!!! note

The ``dependencies`` field must be provided even if empty.

`uv run` will search for and use the required Python version. The Python version will download if it is not installed — see the documentation on Python versions for more details.

Using a shebang to create an executable file

A shebang can be added to make a script executable without using `uv run` — this makes it easy to run scripts that are on your PATH or in the current folder.

For example, create a file called `greet` with the following contents

```
#!/usr/bin/env -S uv run --script

print("Hello, world!")
```

Ensure that your script is executable, e.g., with `chmod +x greet`, then run the script:

```
$ ./greet
Hello, world!
```

Declaration of dependencies is also supported in this context, for example:

```
#!/usr/bin/env -S uv run --script
#
# /// script
# requires-python = ">=3.12"
# dependencies = ["httpx"]
# ///

import httpx

print(httpx.get("https://example.com"))
```

Using alternative package indexes

If you wish to use an alternative package index to resolve dependencies, you can provide the index with the `--index` option:

```
$ uv add --index "https://example.com/simple" --script example.py 'requests<3' 'rich'
```

This will include the package data in the inline metadata:

```
# [[tool.uv.index]]
# url = "https://example.com/simple"
```

If you require authentication to access the package index, then please refer to the package index documentation.

Locking dependencies

uv supports locking dependencies for PEP 723 scripts using the `uv.lock` file format. Unlike with projects, scripts must be explicitly locked using `uv lock`:

```
$ uv lock --script example.py
```

Running `uv lock --script` will create a `.lock` file adjacent to the script (e.g., `example.py.lock`).

Once locked, subsequent operations like `uv run --script`, `uv add --script`, `uv export --script`, and `uv tree --script` will reuse the locked dependencies, updating the lockfile if necessary.

If no such lockfile is present, commands like `uv export --script` will still function as expected, but will not create a lockfile.

Improving reproducibility

In addition to locking dependencies, uv supports an `exclude-newer` field in the `tool.uv` section of inline script metadata to limit uv to only considering distributions released before a specific date. This is useful for improving the reproducibility of your script when run at a later point in time.

The date should be specified as an RFC 3339 timestamp (e.g., `2006-12-02T02:07:43Z`).

```
# /// script
# dependencies = [
#     "requests",
# ]
# [tool.uv]
# exclude-newer = "2023-10-16T00:00:00Z"
# ///
```

```
import requests
```

```
print(requests.__version__)
```

Using different Python versions

uv allows arbitrary Python versions to be requested on each script invocation, for example:

```
import sys
```

```
print(".".join(map(str, sys.version_info[:3])))
```

```
$ # Use the default Python version, may differ on your machine
```

```
$ uv run example.py
```

```
3.12.6
```

```
$ # Use a specific Python version
```

```
$ uv run --python 3.10 example.py
```

```
3.10.15
```

See the Python version request documentation for more details on requesting Python versions.

Using GUI scripts

On Windows uv will run your script ending with .pyw extension using pythonw:

```
from tkinter import Tk, ttk
```

```
root = Tk()
```

```
root.title("uv")
```

```
frm = ttk.Frame(root, padding=10)
```

```
frm.grid()
```

```
ttk.Label(frm, text="Hello World").grid(column=0, row=0)
```

```
root.mainloop()
```

```
PS> uv run example.pyw
```

```
Run Result{: style="height:50px;width:150px"}
```

Similarly, it works with dependencies as well:

```
import sys
```

```
from PyQt5.QtWidgets import QApplication, QWidget, QLabel, QGridLayout
```

```
app = QApplication(sys.argv)
```

```
widget = QWidget()
```

```
grid = QGridLayout()
```

```
text_label = QLabel()
```

```
text_label.setText("Hello World!")
```

```
grid.addWidget(text_label)
```

```
widget.setLayout(grid)
```

```
widget.setGeometry(100, 100, 200, 50)
```

```
widget.setWindowTitle("uv")
```

```
widget.show()
```

```
sys.exit(app.exec_())
```

```
PS> uv run --with PyQt5 example_pyqt.pyw
```

```
Run Result{: style="height:50px;width:150px"}
```

Next steps

To learn more about `uv run`, see the [command reference](#).

Or, read on to learn how to run and install tools with `uv`.